Import Packages

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
```

Load Feature Dataset & P-value Rank

- Results of ML3-1 and ML3-2

In [2]:
```
FeatureData = pd.read_csv(
    "../ML3/data/FeatureData.csv",
    header=None,
)
FeatureData.shape
```

Out[2]: (270, 360)

In [3]:
```
P_value_Rank = pd.read_csv(
    "../ML3/data/P_value_Rank.csv",
    header=None,
)
P_value_Rank
```

Out[3]:

	0	1
0	198.0	4.762393e-81
1	110.0	5.270311e-77
2	170.0	4.615377e-76
3	133.0	5.747675e-74
4	134.0	7.585991e-74
...	...	...
265	85.0	8.999631e-01
266	95.0	9.358453e-01
267	255.0	9.450087e-01
268	217.0	9.695416e-01
269	166.0	9.776721e-01

270 rows × 2 columns

Select Target Features for Demensionality Reduction Based on P-value

In [4]:
```
# Select features from StartRank to StartRank+Num
StartRank = 1
Number = 30

SelectedFeatues = np.zeros((Number, FeatureData.shape[1]))

s = 0
for i in range(StartRank, StartRank + Number):

    index = int(P_value_Rank.iloc[i - 1, 0])
```

```
    SelectedFeatues[s, :] = FeatureData.iloc[index, :].values
    s += 1

FeatureSelected = pd.DataFrame(SelectedFeatues).T
FeatureSelected
```

Out[4]:

	0	1	2	3	4	5	6	7	8	9	...	20
0	1.481424	1.268380	12.777754	0.654730	0.428672	-2.7173	1.620323	0.167877	0.028183	1.265919	...	0.095414
1	1.478799	1.428695	13.280290	0.649638	0.421593	-2.7250	1.597625	0.167090	0.027919	1.299383	...	0.095599
2	1.482299	1.430385	12.905222	0.641463	0.411441	-2.7224	1.625148	0.172784	0.029854	1.281556	...	0.095412
3	1.477326	1.428752	13.079692	0.638104	0.406741	-2.7520	1.575759	0.172377	0.029714	1.303882	...	0.095632
4	1.478828	1.435017	12.862634	0.627408	0.393194	-2.7517	1.564056	0.171432	0.029389	1.295184	...	0.095620
...	...	...	...	...	...	...	...	...	...	...	...	...
355	1.489518	1.433793	12.231166	0.671847	0.450741	-2.8459	1.578616	0.180131	0.032447	1.271762	...	0.095817
356	1.485393	1.478560	11.902660	0.661014	0.436692	-2.8570	1.540658	0.184310	0.033970	1.269566	...	0.095778
357	1.486447	1.478878	11.811784	0.681616	0.464223	-2.8318	1.588620	0.185856	0.034543	1.275704	...	0.095855
358	1.489181	1.470033	12.339840	0.672230	0.451389	-2.8524	1.522793	0.171050	0.029258	1.273485	...	0.095829
359	1.487358	1.483164	11.875705	0.665655	0.442804	-2.8602	1.530078	0.176170	0.031036	1.273770	...	0.096117

360 rows × 30 columns

Standardization of features

In [5]:
```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

FeatureSelected_std = StandardScaler().fit_transform(FeatureSelected)
pd.DataFrame(FeatureSelected_std)
```

Out[5]:

	0	1	2	3	4	5	6	7	8	9	...	
0	-0.776847	-3.420112	0.469637	-0.171224	-0.180105	0.773818	0.726666	-1.405927	-1.381434	-1.841564	...	-0
1	-1.526489	-0.292716	1.311470	-0.381671	-0.402063	0.708676	0.352155	-1.512154	-1.480919	1.233430	...	-0
2	-0.526880	-0.259748	0.683167	-0.719494	-0.720387	0.730672	0.806293	-0.743554	-0.750557	-0.404665	...	-0
3	-1.947166	-0.291613	0.975434	-0.858271	-0.867777	0.480255	-0.008646	-0.798420	-0.803505	1.646881	...	-0
4	-1.518389	-0.169397	0.611825	-1.300261	-1.292566	0.482793	-0.201750	-0.926091	-0.926229	0.847654	...	-0
...	...	...	...	...	...	...	...	...	...	...	...	
355	1.534911	-0.193261	-0.445990	0.536100	0.511937	-0.314142	0.038488	0.248310	0.228124	-1.304660	...	0
356	0.356688	0.680043	-0.996295	0.088451	0.071388	-0.408049	-0.587832	0.812514	0.803010	-1.506446	...	0
357	0.657705	0.686250	-1.148526	0.939807	0.934670	-0.194856	0.203567	1.021315	1.019108	-0.942348	...	0
358	1.438797	0.513686	-0.263944	0.551953	0.532235	-0.369132	-0.882609	-0.977644	-0.975592	-1.146249	...	0
359	0.918031	0.769842	-1.041449	0.280238	0.263051	-0.435121	-0.762402	-0.286439	-0.304580	-1.120142	...	1

360 rows × 30 columns

t-SNE Implementation

In [7]:
```
from sklearn.manifold import TSNE  # Import a package for t-SNE

dim = 2
time_start = time.time()
tsne = TSNE(
    n_components=dim, verbose=1, perplexity=50, max_iter=500, random_state=1
)
# 'perplexity' is generally set to a value between 5 and 50.
tsne_results = tsne.fit_transform(FeatureSelected_std)

print(
    "\n\n t-SNE done! Time elapsed: {} seconds\n\n".format(
        time.time() - time_start
    )
)

pd.DataFrame(tsne_results)
```

```
[t-SNE] Computing 151 nearest neighbors...
[t-SNE] Indexed 360 samples in 0.000s...
[t-SNE] Computed neighbors for 360 samples in 1.369s...
[t-SNE] Computed conditional probabilities for sample 360 / 360
[t-SNE] Mean sigma: 1.880729
[t-SNE] KL divergence after 250 iterations with early exaggeration: 44.314762
[t-SNE] KL divergence after 500 iterations: 0.467883


t-SNE done! Time elapsed: 1.5359933376312256 seconds
```

Out[7]:

	0	1
0	-9.412018	-2.728613
1	-17.529345	-1.848591
2	-13.687891	-1.607496
3	-15.346423	-1.008596
4	-14.246557	3.418501
...	...	...
355	16.925842	2.356761
356	16.922365	1.785547
357	18.241198	-2.110587
358	11.388663	3.135637
359	13.101482	2.840780

360 rows × 2 columns

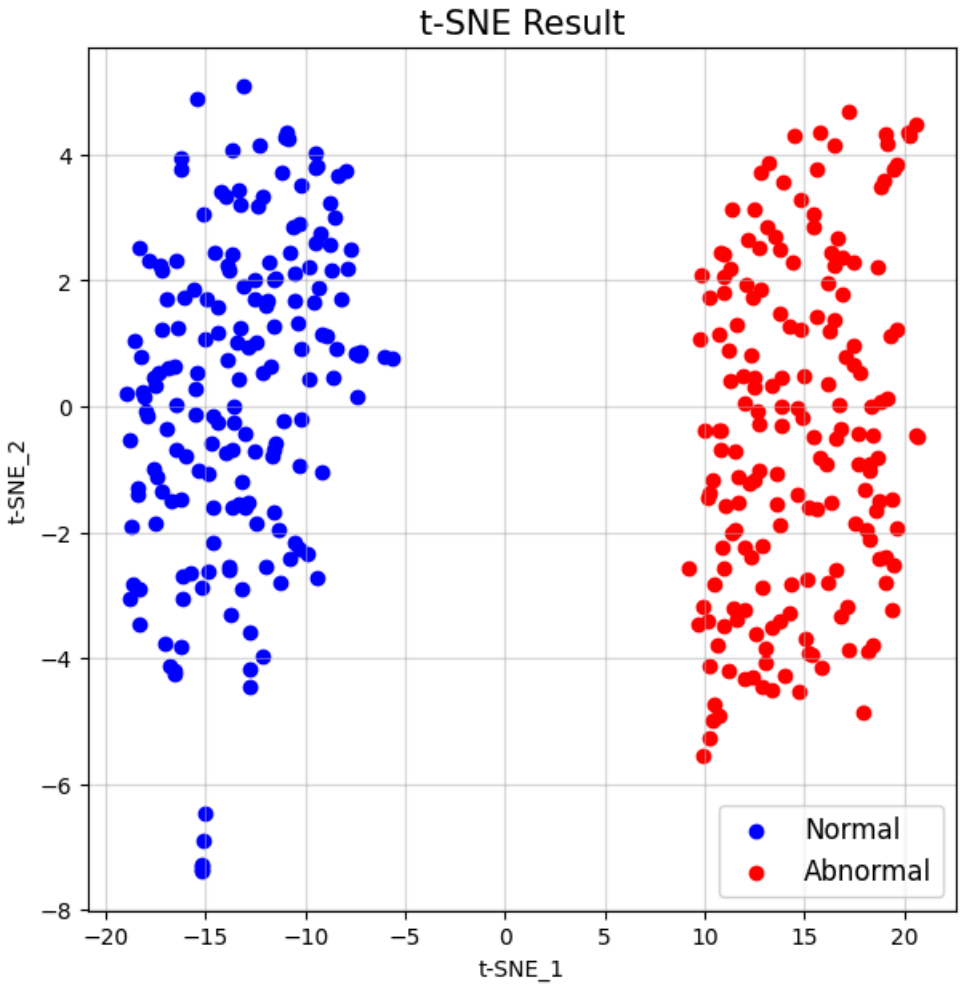
In [8]:
```
NoOfData = int(FeatureSelected.shape[0] / 2)

plt.figure(figsize=(7, 7))

plt.scatter(
    tsne_results[:NoOfData, 0],
    tsne_results[:NoOfData, 1],
    marker="o",
    label="Normal",
    c="b",
)
plt.scatter(
    tsne_results[NoOfData:, 0],
    tsne_results[NoOfData:, 1],
    marker="o",
```

```
    label="Abnormal",
    c="r",
)
plt.title("t-SNE Result", fontsize=15)
plt.grid(alpha=0.5)
plt.legend(fontsize=12)
plt.xlabel("t-SNE_1")
plt.ylabel("t-SNE_2")

plt.show()
```



Comparison with the t-SNE result of bottom 30 features (p-value)

In [11]:
```
StartRank = 240
Number = 30

SelectedFeatues = np.zeros((Number, FeatureData.shape[1]))

s = 0
for i in range(StartRank, StartRank + Number):
    index = int(P_value_Rank.iloc[i - 1, 0])
    SelectedFeatues[s, :] = FeatureData.iloc[index, :].values
    s += 1

FeatureSelected_std = StandardScaler().fit_transform(
    pd.DataFrame(SelectedFeatues).T
)

dim = 2
tsne = TSNE(
    n_components=dim, verbose=1, perplexity=50, max_iter=500, random_state=1
```

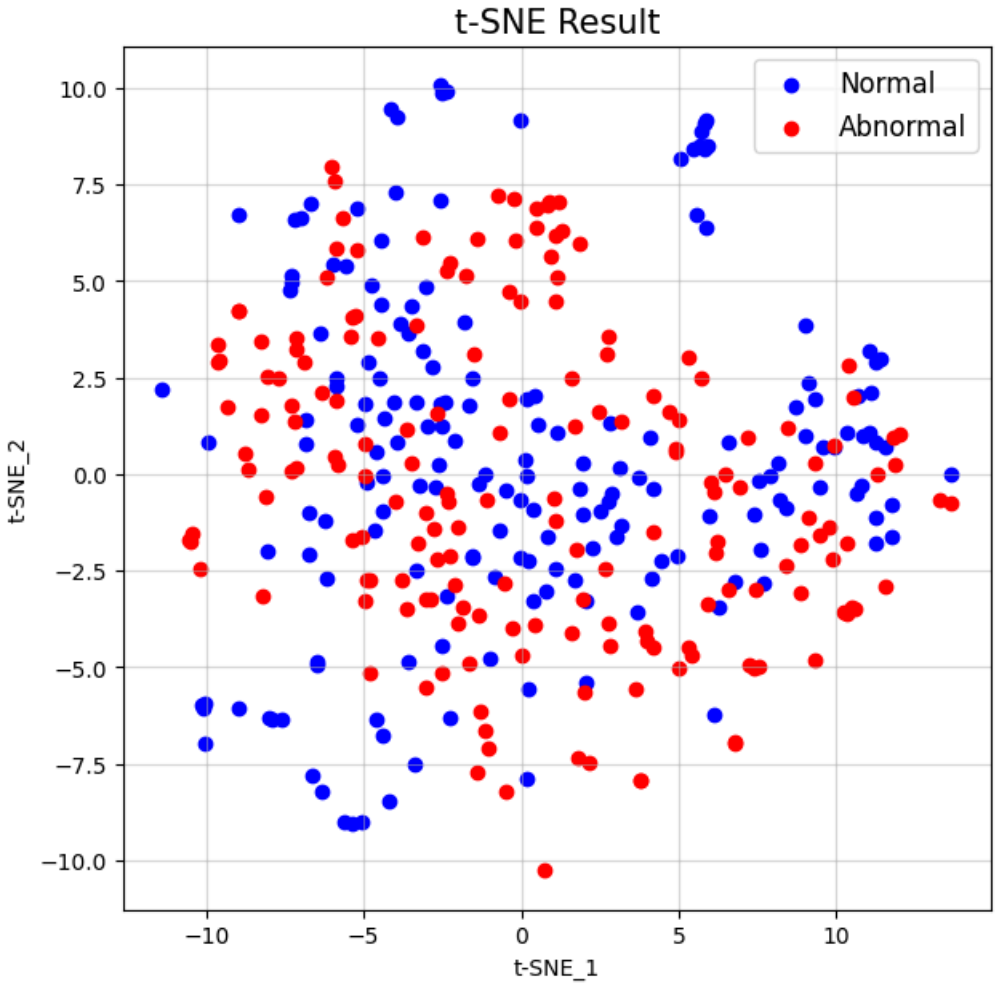
```
)
tsne_results = tsne.fit_transform(FeatureSelected_std)


plt.figure(figsize=(7, 7))

plt.scatter(
    tsne_results[:NoOfData, 0],
    tsne_results[:NoOfData, 1],
    marker="o",
    label="Normal",
    c="b",
)
plt.scatter(
    tsne_results[NoOfData:, 0],
    tsne_results[NoOfData:, 1],
    marker="o",
    label="Abnormal",
    c="r",
)
plt.title("t-SNE Result", fontsize=15)
plt.grid(alpha=0.5)
plt.legend(fontsize=12)
plt.xlabel("t-SNE_1")
plt.ylabel("t-SNE_2")

plt.show()
```

```
[t-SNE] Computing 151 nearest neighbors...
[t-SNE] Indexed 360 samples in 0.000s...
[t-SNE] Computed neighbors for 360 samples in 0.012s...
[t-SNE] Computed conditional probabilities for sample 360 / 360
[t-SNE] Mean sigma: 1.989063
[t-SNE] KL divergence after 250 iterations with early exaggeration: 50.710171
[t-SNE] KL divergence after 500 iterations: 0.766504
```


t-SNE Result
Normal
Abnormal
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
t-SNE_2
−10
−5
0
5
10
t-SNE_1